LECTURE

State Space Models and the Kalman Filter

From latent states to EKF, UKF and particle filters

Structure · Kalman filter · ARMA in state space · ML estimation · RLS · Nonlinear filters · Finance

How to estimate a hidden state from noisy, incomplete observations —
and how the filter generalizes when the world is nonlinear and non-Gaussian.

01

PART 1

State Space Model

Latent dynamics, measurement, and the link to ARIMA

SSMs estimate a hidden state, not just a value

Motivation — traditional time series vs. State Space Model (SSM)

Traditional time series	State Space Model
Forecast the future from past observations	Forecast while estimating a latent state
ARIMA structure is fixed	System model and observation model are separated
The measurement is everything	Noise and measurement error are modeled
Vulnerable to missing values	Robust to missing values (KF skips the update)
Static parameters	Supports time-varying (adaptive) parameters

An SSM pairs a state and an observation equation

Structure of the state space model — the hidden state drives what we measure

State and observation equations

System (latent dynamics)

$$X_t = AX_{t-1} + Bu_{t-1} + e_{1,t}$$

Observation (measurement)

$$Y_t = CX_t + e_{2,t}$$

Features

X_t : latent state (m-dimensional)

Y_t : observation vector

$e_1,\ e_2$: system / measurement noise

Noises are assumed to be independent white noise

A falling body is the classic two-state example

Representative example ① — Falling Body; only the position can be observed

State definition

$$x_1(t) = z(t) \quad \text{(height)}, \qquad x_2(t) = z'(t) \quad \text{(velocity)}$$

Only the position is observable

$$Y_t = x_1(t) + e_{2,t}$$

Discretization

$$X_t = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} X_{t-1} + \begin{pmatrix} -1/2 \\ -1 \end{pmatrix} g + e_{1,t}$$

$$Y_t = (1\ 0) X_t + e_{2,t}$$

Reading the model

- State: height and velocity
- Velocity is never measured
- Gravity g is a known input
- The filter infers velocity from positions

A noisy random walk is the simplest SSM

Analyze a simple SSM to see the series' structure and its link to ARIMA

Model definition	Equation	Description
State equation (Random Walk)	$X_t = X_{t-1} + \eta_t$	System noise (white noise)
Observation equation	$Y_t = X_t + \varepsilon_t$	Observation noise (white noise)

System noise moves the state; observation noise blurs what we see.

A noisy random walk is an ARIMA(0,1,1) process

Identify the ARIMA structure through the first difference of the observed process

1. Apply the difference operator

$$\nabla Y_t = Y_t - Y_{t-1}$$

$$\nabla Y_t = (X_t + \varepsilon_t) - (X_{t-1} + \varepsilon_{t-1})$$

2. Substitute the random walk

$$\nabla Y_t = \eta_t + \varepsilon_t - \varepsilon_{t-1}$$

3–4. ACF and conclusion

- The difference has a significant autocorrelation only at lag 1
- That is the typical signature of MA(1)
- Differenced once it follows MA(1), so the observed process is ARIMA(0,1,1)

02

PART 2

Kalman Filter

Prediction and update, ARMA in state space, likelihood, adaptive estimation

The Kalman filter predicts, then corrects

Kalman Filter — core concepts

Step	Description
Prediction	Predict the future state from the current state
Update / Reconstruction	Update the state with the observation, reducing the error
Kalman Gain	The weight that sets trust in the observation vs. the model

Predict → observe → correct → repeat, one time step at a time

Five equations define the Kalman filter

Kalman Filter — update equations

Prediction

$$\hat{X}_{t|t-1} = A\hat{X}_{t-1|t-1} + Bu_{t-1}$$

$$P_{t|t-1} = AP_{t-1|t-1}A^T + \Sigma_1$$

Update

$$K_t = P_{t|t-1}C^T(CP_{t|t-1}C^T + \Sigma_2)^{-1}$$

$$\hat{X}_{t|t} = \hat{X}_{t|t-1} + K_t(Y_t - C\hat{X}_{t|t-1})$$

$$P_{t|t} = (I - K_tC)P_{t|t-1}$$

The Kalman gain decides how far the new observation pulls the prediction.

Any ARMA(p, q) model has a state space form

ARMA to SSM — the ARMA(p, q) model, the state vector and the observation equation

$$Y_t - \varphi_1 Y_{t-1} - \ldots - \varphi_p Y_{t-p} = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \ldots + \theta_q \varepsilon_{t-q}$$

1. State vector (dim. p + q)

$$X_t = \begin{bmatrix} Y_t \\ Y_{t-1} \\ \vdots \\ Y_{t-p+1} \\ \varepsilon_t \\ \varepsilon_{t-1} \\ \vdots \\ \varepsilon_{t-q+1} \end{bmatrix}$$

Observation equation

$$Y_t = HX_t, \qquad H = \begin{bmatrix} 1 & 0 & \cdots & 0 \end{bmatrix}$$

State-space form

$$\begin{aligned} X_t &= GX_{t-1} + Rv_t, \quad v_t \sim N(0, \sigma^2) \\ Y_t &= HX_t, \quad H = (1, 0, \ldots, 0) \end{aligned}$$

The transition matrix stores the ARMA coefficients

2. State transition equation — define a new noise, and the state equation follows

New noise

$$v_t = \varepsilon_t, \qquad v_t \sim N(0, \sigma^2)$$

State equation

$$X_t = GX_{t-1} + Rv_t$$

where

$$G = \begin{bmatrix} \varphi_1 & \varphi_2 & \cdots & \varphi_{p-1} & \varphi_p & \theta_1 & \theta_2 & \cdots & \theta_q \\ 1 & 0 & \cdots & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & 0 & 0 & \cdots & 0 \\ \hline 0 & 0 & \cdots & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & 1 & 0 & \cdots & 0 \\ \vdots & & & & \vdots & \vdots & \ddots & & \vdots \\ 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix}$$

$$R = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

The filter keeps running through missing data

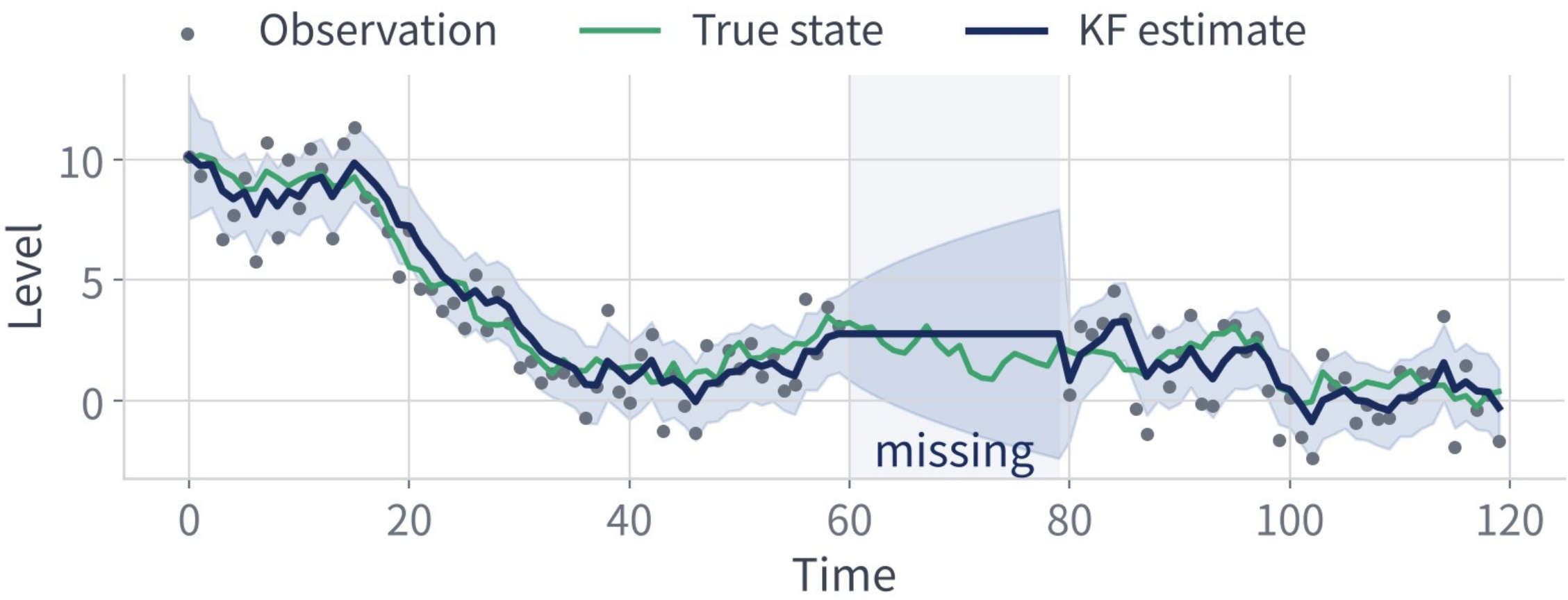
Missing-value handling — the KF works even with missing time points

Observation available	Observation missing
Prediction + Update	Prediction only
Uncertainty decreases	Uncertainty increases

No observation → nothing to update with → the prediction is carried forward

Uncertainty widens in a gap, collapses after it

Local-level model (random walk + noise); shaded band = ±2 s.d. · ※ simulated data



The filter's prediction errors give the likelihood

ML estimation (Likelihood via Kalman Filter) — using KF prediction errors

$$\tilde{Y}_t = Y_t - \hat{Y}_{t|t-1}$$

$$\log L = -\frac{1}{2}\sum_{t=1}^{N}(\log\det\Sigma_{yy,t|t-1} + \tilde{Y}_t^T\Sigma_{yy,t|t-1}^{-1}\tilde{Y}_t)$$

Write ARMA in state space and run the Kalman filter

- Prediction errors (innovations) form a Gaussian log-likelihood
- Maximizing it estimates AR/MA coefficients and noise variances
- The method works even when observations are missing

RLS updates regression coefficients online

Recursive & Adaptive Estimation (Kalman + RLS) — core idea of RLS (Recursive Least Squares)

Core idea

- Update β as soon as new data arrive — online estimation without recomputing all past data
- Combine the previous β estimate with the information in the current data
- Use the inverse correlation matrix to reduce correlation → faster convergence; faster and more stable than LMS

Core equations

$$\beta_t = \beta_{t-1} + K_t(y_t - x_t^T \beta_{t-1})$$

$$K_t = \frac{P_{t-1} x_t}{\lambda + x_t^T P_{t-1} x_t}$$

$$P_t = \frac{1}{\lambda}\left(P_{t-1} - K_t x_t^T P_{t-1}\right)$$

The forgetting factor λ sets how fast RLS adapts

λ is the lever that tunes adaptivity to a fast-changing environment

λ value	Meaning	Result
Close to 1	Old data trusted about as much as new data	β almost fixed (little change)
Below 1	Recent data weighted more	β changes fast (sensitive to market shifts)

Fixed λ

- Gradual forgetting

Variable λ

- Adapts fast when the environment changes sharply

A basic Kalman filter assumes slow change

Stable system → KF vs. fast-changing system → Adaptive RLS with λ below 1

$$\beta_t = \beta_{t-1} + w_t, \qquad w_t \sim N(0, Q)$$

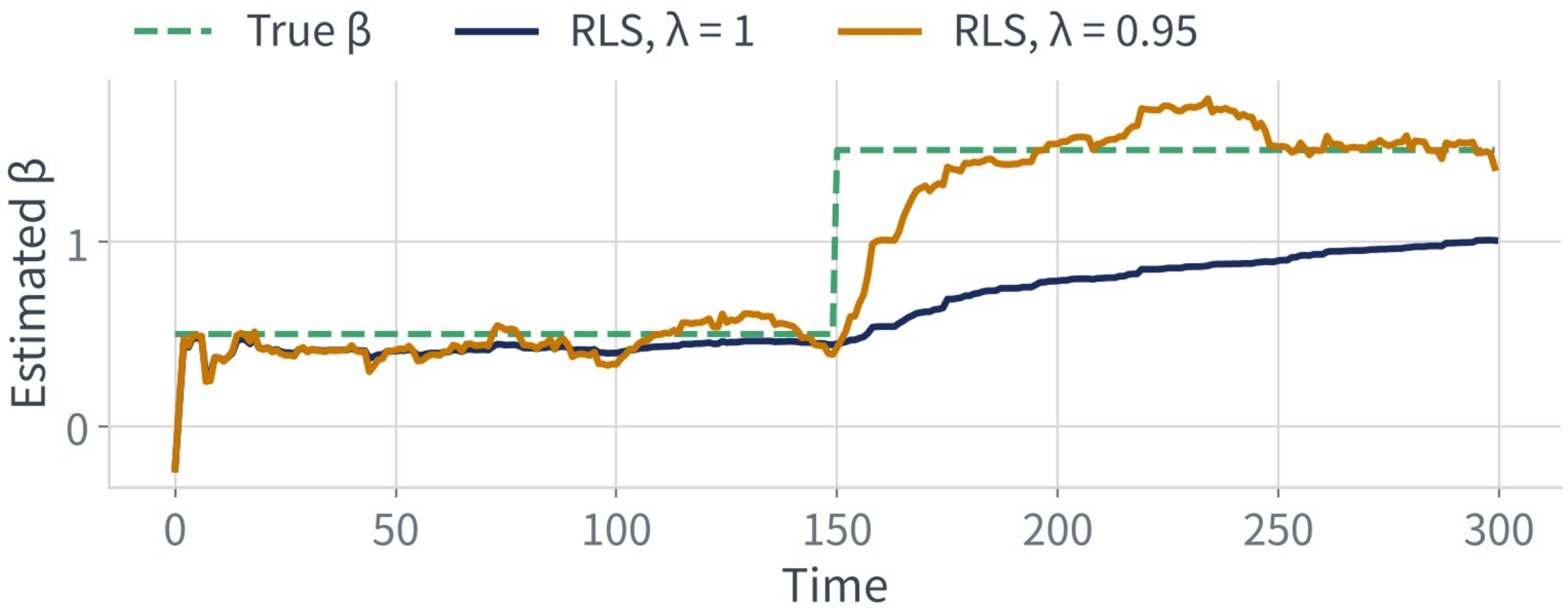
KF state equation — Q is the noise (size of change) of the state

Q	System behavior
Zero	State barely changes (almost fixed)
Small	Slowly changing system (smooth dynamics)
Large	Beta fluctuates a lot (strongly time-varying)

Basic KF is optimal for gradual change → reacts slowly to abrupt shifts (small gain)

A forgetting factor below 1 catches a sudden beta shift

RLS on a regression whose β jumps from 0.5 to 1.5 at t = 150 · ※ simulated data



The data problem decides which model to use

Practical connection — when to use which model

Situation	Suitable model
Physical systems / dynamics	State Space + KF
Estimating ARIMA structure	ARMA → SS + KF-ML
Noisy sensor data	SS + KF
Missing observations	SS + KF
Time-varying parameters	Adaptive KF / Adaptive RLS
Nonlinear systems	EKF / UKF / Particle Filter

A missing observation sets the gain to zero

Practical KF: initialization and missing data — the missing-value mechanism

Kalman Gain

With no observation, the observation variance is treated as infinite, so the Kalman gain becomes 0.

State estimate

With zero gain, the reconstructed estimate is the previous prediction — no new information.

reconstructed = previous estimate

Variance

Without new information, the variance of the state estimate does not decrease (uncertainty is kept).

Observation missing → prediction only; the update step is skipped

Initial uncertainty matters more than the initial value

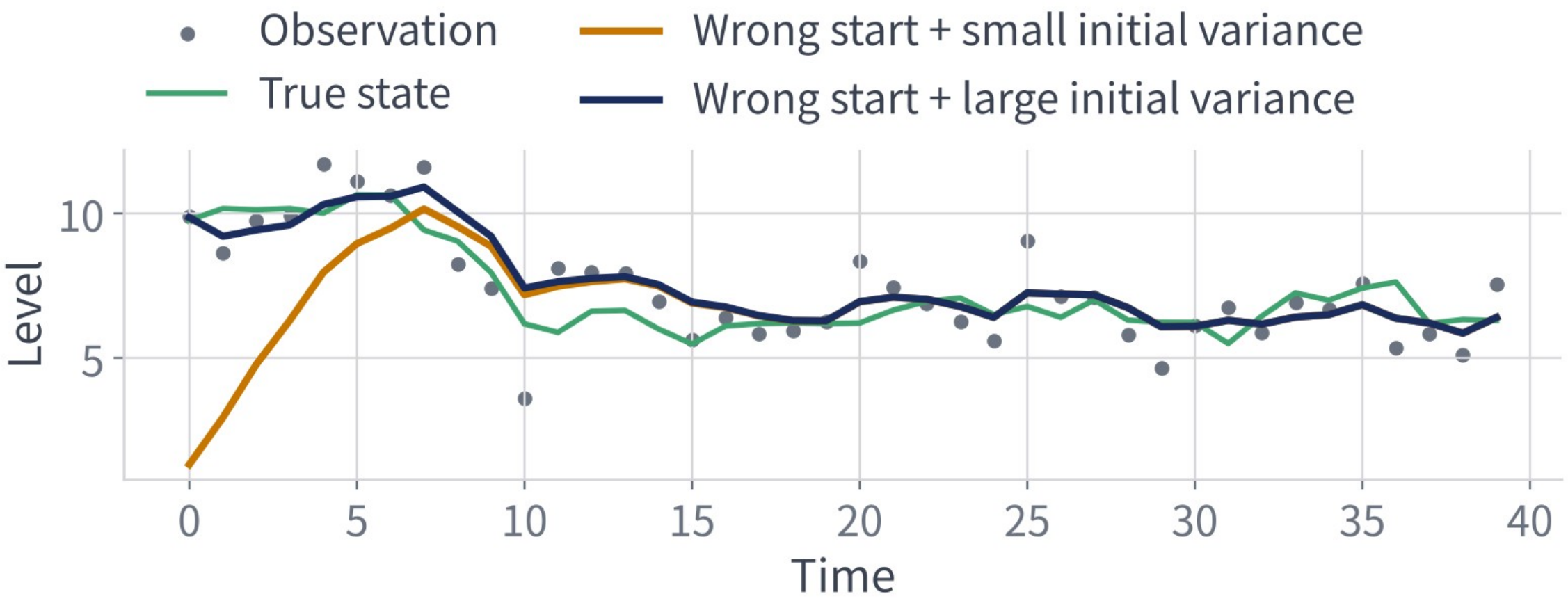
Convergence speed and performance depend on how appropriately the initial uncertainty is set

Scenario	Initial state & variance	Key result
Wrong start + overconfidence	Inaccurate value + low initial variance	Slow convergence and overshooting
Wrong start + proper uncertainty	Inaccurate value + large initial variance	Recognizes uncertainty; converges very fast on the first observation

Starting in the wrong place is not a big problem — but the uncertainty attached to that estimate must be appropriate.

A large initial variance lets the filter forget a bad start

Same wrong starting value (0), two initial variances · ※ simulated data



Uncertainty grows without data, shrinks with data

Filter performance when data are missing

No observations

Driven by system noise, the uncertainty of the prediction interval grows step by step.

New observation

Uncertainty drops quickly and the state estimate is corrected.

Caution

Overestimating the system noise can make uncertainty explode while observations are missing.

System noise sets how fast uncertainty grows while data are missing

03

PART 3

Advanced Filters — EKF, UKF, Particle Filter

Extend KF filtering to nonlinear systems and to tracking the whole distribution

Real systems break the KF's linear-Gaussian world

The KF needs (1) a linear model (2) Gaussian noise (3) linear, fixed transforms A and C — but reality...

Example	Characteristic
Robots / drones / SLAM	Rotation and coordinate transforms → nonlinear
Real-time trading strategies	Market jumps, non-normal volatility structure
Radar / sensing	Nonlinear sensor models
Neural / biological data	Non-Gaussian noise

Linear + Gaussian KF hits its limit → EKF → UKF → Particle Filter

The EKF linearizes, then runs the Kalman filter

1. Extended Kalman Filter (EKF) — extends the KF by linearization; f and h are nonlinear

Model

$$X_t = f(X_{t-1}) + e_{1,t}$$

$$Y_t = h(X_t) + e_{2,t}$$

Core idea — Jacobians replace A and C

$$A_t = \frac{\partial f}{\partial X}|_{X=\hat{X}_{t-1|t-1}}, \quad C_t = \frac{\partial h}{\partial X}|_{X=\hat{X}_{t|t-1}}$$

1st-order Taylor expansion; all other equations are the same as the KF

The EKF is fast but weak under strong nonlinearity

1. Extended Kalman Filter (EKF) — advantages and disadvantages

Advantages	Disadvantages
Fast computation	Poor approximation under strong nonlinearity
Efficient in control / SLAM	Jacobian must be computed
Widely used in industry	Can distort the distribution

When is it suitable?

- Nonlinear but not changing sharply — robots, aerospace, SLAM, sensor fusion

The UKF propagates the distribution itself

2. Unscented KF (UKF) — born to fix the EKF's linearization error; no linearization, only sigma points

Core idea

- Represent the state distribution by a small set of deterministically chosen points
- Keep the mean and covariance of the distribution
- Place 2n + 1 representative points of a multivariate Gaussian

Algorithm flow

1. Generate the sigma points
2. Pass each point through f and h
3. Combine the mean / covariance of the result points into a new distribution

The UKF is more accurate than the EKF at a higher cost

2. Unscented Kalman Filter (UKF) — advantages and disadvantages

Advantages	Disadvantages
Much more accurate than EKF	More computation
No Jacobian needed	Still limited under extreme non-normality
Stable under strong nonlinearity	Sigma-point count grows in high dimensions

Suitable cases

- Strongly nonlinear sensors, radar, safety control; multi-sensor fusion, inertial navigation

EKF vs UKF in one line

- EKF = linear approximation of the nonlinear function
- UKF = pass the distribution directly

The particle filter assumes no distribution shape

3. Particle Filter (Sequential Monte Carlo) — the most general and powerful Bayesian filter

Core concept

- Represent the state distribution by a set of particles (samples)
- Each particle = one possible state
- Update particle weights when an observation arrives
- Keep high-probability particles, drop low ones (resampling)

Algorithm

1. Sample many particles from the initial distribution
2. Move each particle by the model (prediction)
3. Update weights by comparing with the observation
4. Resample to suppress degeneracy
5. Repeat

Particle filters fit any distribution, at a cost

3. Particle Filter (Sequential Monte Carlo) — advantages and disadvantages

Advantages	Disadvantages
Works for any nonlinear / non-normal system	Heavy computation
Can represent multi-modal distributions	Curse of dimensionality in high dimensions
Robust to jumps and regime shifts	Collapses when particles are too few

Suitable cases

- Finance: regime switching, jump models, volatility models · Radar / SLAM / tracking · Non-Gaussian dynamics

The four filters trade generality against computation

Core comparison of the methods

Model	Distribution	Nonlinearity	Complexity	Practice
KF	Gaussian	None	Low	Sensors, time series
EKF	Gaussian	Linearization	Medium	Robots, aerospace
UKF	Gaussian	Sigma points	Medium–high	Autonomous driving, sensor fusion
Particle Filter	Unrestricted	Sample based	High	Finance, equipment, regime change

Start simple; move up only when data demand it

Which filter in practice — recommendation by situation

Situation	Recommendation
Linear system + Gaussian	KF
Weak nonlinearity + computing constraints	EKF
Strong nonlinearity + Jacobian hard to compute	UKF
Jumps, regimes, non-normality, multiple modes	Particle Filter
“I don't know what's best”	UKF → PF if performance falls short

Each filter carries the distribution differently

Conclusion — one-sentence summary

Name	Core
EKF	“Linearize the nonlinearity first, then apply the KF”
UKF	“Pass the distribution through the function, preserving mean and covariance”
PF	“Represent the full state distribution by particles and update it”

A1

APPENDIX 1

EKF, UKF and PF — Equation Summary

Equation-centered core summary of the three filters

The EKF re-linearizes at every step

1. Extended Kalman Filter (EKF) — model, linearization, prediction and update

Model

$$X_t = f(X_{t-1}) + e_{1,t}, \qquad e_{1,t} \sim \mathcal{N}(0, \Sigma_1)$$

$$Y_t = h(X_t) + e_{2,t}, \qquad e_{2,t} \sim \mathcal{N}(0, \Sigma_2)$$

(1) Linearization (Jacobian)

$$A_t = \left.\frac{\partial f}{\partial X}\right|_{X=\hat{X}_{t-1|t-1}}, \qquad C_t = \left.\frac{\partial h}{\partial X}\right|_{X=\hat{X}_{t|t-1}}$$

(2) Prediction

$$\hat{X}_{t|t-1} = f(\hat{X}_{t-1|t-1})$$

$$P_{t|t-1} = A_t P_{t-1|t-1} A_t^T + \Sigma_1$$

(3) Update

$$K_t = P_{t|t-1} C_t^T \left(C_t P_{t|t-1} C_t^T + \Sigma_2\right)^{-1}$$

$$\hat{X}_{t|t} = \hat{X}_{t|t-1} + K_t \left(Y_t - h(\hat{X}_{t|t-1})\right)$$

$$P_{t|t} = (I - K_t C_t) P_{t|t-1}$$

The UKF places 2n + 1 sigma points

2. Unscented Kalman Filter (UKF) — model (same) and sigma point generation for state dimension n

$$X_t = f(X_{t-1}) + e_{1,t}, \qquad Y_t = h(X_t) + e_{2,t}$$

Core: sigma point generation

$$\chi_{t-1}^{(0)} = \hat{X}_{t-1|t-1}$$

$$\chi_{t-1}^{(i)} = \hat{X}_{t-1|t-1} + \left(\sqrt{(n+\xi)P_{t-1|t-1}}\right)_i, \quad i = 1, \ldots, n$$

$$\chi_{t-1}^{(i+n)} = \hat{X}_{t-1|t-1} - \left(\sqrt{(n+\xi)P_{t-1|t-1}}\right)_i, \quad i = 1, \ldots, n$$

Prediction averages the propagated sigma points

2. Unscented Kalman Filter (UKF) — (1) Prediction

Pass each sigma point through the function

$$\chi_{t|t-1}^{(i)} = f(\chi_{t-1}^{(i)})$$

State prediction by weighted average

$$\hat{X}_{t|t-1} = \sum_i w_m^{(i)} \chi_{t|t-1}^{(i)}$$

Covariance

$$P_{t|t-1} = \sum_i w_c^{(i)} (\chi_{t|t-1}^{(i)} - \hat{X}_{t|t-1})(\chi_{t|t-1}^{(i)} - \hat{X}_{t|t-1})^T + \Sigma_1$$

The update builds covariances from the points

2. Unscented Kalman Filter (UKF) — (2) Update

Observation sigma points

$$\gamma^{(i)} = h(\chi_{t|t-1}^{(i)})$$

Observation mean

$$\hat{Y}_{t|t-1} = \sum_i w_m^{(i)} \gamma^{(i)}$$

Observation covariance and cross covariance

$$S = \sum_i w_c^{(i)} (\gamma^{(i)} - \hat{Y}_{t|t-1})(\gamma^{(i)} - \hat{Y}_{t|t-1})^T + \Sigma_2$$

$$P_{xy} = \sum_i w_c^{(i)} (\chi_{t|t-1}^{(i)} - \hat{X}_{t|t-1})(\gamma^{(i)} - \hat{Y}_{t|t-1})^T$$

The UKF gain comes from the two covariances

2. Unscented Kalman Filter (UKF) — (2) Update, continued

Kalman gain

$$K_t = P_{xy} S^{-1}$$

State update

$$\hat{X}_{t|t} = \hat{X}_{t|t-1} + K_t(Y_t - \hat{Y}_{t|t-1})$$

Covariance

$$P_{t|t} = P_{t|t-1} - K_t S K_t^T$$

Neural networks can serve as f and h

2. Unscented Kalman Filter (UKF) — [Reference] examples of f and h in the deep learning world

Field	f	h
Financial time series	LSTM	MLP
High-frequency market microstructure	Transformer	CNN
Option price inversion	Neural ODE	Black–Scholes
Volatility tracking	GRU	MLP
Regime-switch detection	RNN	Softmax classifier
Autonomous driving	LSTM	CNN (camera)
Brain signals	RNN	EEG CNN

Importance sampling yields weighted particles

3. Particle Filter (SMC) — goal: estimate the posterior as a distribution of arbitrary shape

Goal

$$p(X_t|Y_{1:t})$$

(1) Importance Sampling

$$X_t^{(i)} \sim q(X_t|X_{t-1}^{(i)}, Y_t)$$

(2) Weight update

$$w_t^{(i)} \propto w_{t-1}^{(i)} \cdot \frac{p(Y_t|X_t^{(i)})\, p(X_t^{(i)}|X_{t-1}^{(i)})}{q(X_t^{(i)}|X_{t-1}^{(i)}, Y_t)}$$

Normalization

$$\tilde{w}_t^{(i)} = \frac{w_t^{(i)}}{\sum_j w_t^{(j)}}$$

Resample when the effective sample size falls too low

3. Particle Filter (SMC) — (3) resampling (an essential step) and (4) state estimation

(3) Resampling

Effective sample size

$$N_{\text{eff}} = \frac{1}{\sum_i (\tilde{w}_t^{(i)})^2}$$

$$N_{\text{eff}} < \text{threshold} \;\Rightarrow\; \text{perform resampling}$$

(4) State estimation

$$\hat{X}_{t|t} = \sum_i \tilde{w}_t^{(i)} X_t^{(i)}$$

One particle cloud filters, forecasts and smooths

3. Particle Filter (SMC) — (5) uses of the PF

(a) Online filtering

- Real-time estimation as each new observation arrives

(b) Forecasting

$$p(X_{t+k}|Y_{1:t}) \approx \{X_{t+k}^{(i)}\}_i$$

(c) Smoothing

$$p(X_t|Y_{1:T})$$

Typical uses

- Finance, radar, autonomous driving, robots

Future

- Propagate particles several steps → predict the future distribution

Past

- Backward smoothing — PF version of RTS smoothing

A2

APPENDIX 2

Particle Filters for Financial Time Series

State design, model example, algorithm and outputs

Financial data play to the particle filter's strengths

Financial data have the following traits, which make the PF the most powerful choice

Financial reality	Weakness of the KF family	PF strength
Regime switching	Linear / normal assumptions break	Regimes can be represented as particles
Fat tails / jumps	Gaussian fails	Any distribution is OK
Volatility clustering	Time-varying variance	Particles reflect it automatically
Non-stationary series	ARIMA insufficient	Model the state process directly

Returns are observed; the market state is hidden

1. Typical state-variable design for a financial particle filter — representative components

State (at time t)	Meaning
μ	Expected return
σ	Volatility
s	Regime (up / down / crisis)
κ	Jump intensity
v	Diffusion factor (e.g., GARCH)
ρ	Correlation transition

Observation = prices / returns; state = the market's hidden structure

Regime-switching GARCH with jumps suits the PF

2. State space example (PF application) — regime switching + GARCH-type volatility + jumps

State

$$X_t = \begin{pmatrix} \mu_t \\ \sigma_t^2 \\ s_t \end{pmatrix}$$

Dynamics

$$\mu_t = \mu_{t-1} + \eta_{\mu,t}$$

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2 + \eta_{\sigma,t}$$

$$P(s_t = j \mid s_{t-1} = i) = \Pi_{ij}$$

Observation

$$r_t = \mu_t + \sigma_t \epsilon_t + J_t q_t$$

Jump occurrence

$$J_t \sim \text{Bernoulli}(p_t)$$

Mixtures, regimes and jumps are hard for EKF and UKF

2. State space example (PF application) — observation equation with jump component

Observation equation

$$r_t = \mu_t + \sigma_t \epsilon_t + J_t q_t$$

Jump occurrence

$$J_t \sim \text{Bernoulli}(p_t)$$

→ Hard to handle with EKF / UKF

- Non-normal
- Mixture distributions
- Discrete regimes
- Conditional variance
- Jumps
- → The PF can handle these directly

The financial PF loop ends in position sizing

3. Financial PF algorithm flow — initialize N particles, then predict, weight, resample, act

Initialize N particles

$$\{X_0^{(i)},\ w_0^{(i)} = 1/N\}, \quad i = 1..N$$

State prediction

$$X_t^{(i)} \sim p(X_t | X_{t-1}^{(i)})$$

Observation-based weights

$$w_t^{(i)} \propto w_{t-1}^{(i)}\, p(r_t | X_t^{(i)})$$

Resample when needed

$$\mathrm{ESS} < c \Rightarrow \text{resample}$$

Use for return forecasts / VaR / rebalancing

$$\hat{\mu}_t,\ \hat{\sigma}_t \ \Rightarrow\ \text{adjust position size}$$

The PF excels at detecting risk, not returns

4. Advantages in practice — what the PF provides

Application	What the PF provides
Market forecasting	Regime probabilities, volatility forecasts
Risk	Scenario-based distribution forecasts
Portfolio	Dynamic weights based on the μ / σ posterior
Option hedging	Reflects non-normal tails
Event response	Real-time estimate of jump risk

Tail · Regime · Volatility — all based on the posterior

Three PF outputs are used most in practice

5. The most useful forms of financial PF output

(1) Regime probability

$$P(s_t = \text{risk-off} | Y_{1:t})$$

Use

- VIX, rate shocks, option delta-hedge timing

(2) Predictive volatility

$$P(\sigma_{t+k} | Y_{1:t})$$

Use

- Variance targeting / position sizing

(3) Scenario-based ES

$$ES_\alpha = E[-r_t \mid r_t < -\text{VaR}_\alpha]$$

Use

- CVaR-based portfolio optimization